Crucible — Theory Modules Architecture

> *Reference document for all five theory modules in the Crucible simulation engine. Each theory is a plugin that runs inside the SimRunner tick loop. Target audience: developers implementing theory stubs in Week 2.*
>
> ***Reading this document should make reading the original papers optional.***

Overview

```
┌──────────────────────────────────────────────────────────────────┐
│  SimRunner tick loop                                              │
│                                                                   │
│  env: dict[str, float]   ←── shared normalized [0,1] state       │
│      │                                                            │
│      ├── apply_shocks(env)                                        │
│      │                                                            │
│      ├── for agent in agents:                                     │
│      │       agent.observe(env) → update beliefs                  │
│      │       agent.deliberate() → list[Action]                    │
│      │       action_deltas = resolve_actions(actions)             │
│      │       env = merge(env, action_deltas)                      │
│      │                                                            │
│      ├── for theory in theories:   ← sorted by priority           │
│      │       delta = theory.update(env, agents, tick)   ← THIS FILE │
│      │       env = merge(env, delta)                              │
│      │                                                            │
│      └── record_metrics(env, tick)                                │
│                                                                   │
│  Theories active in Crucible v1:                                  │
│    Priority 0: richardson_arms_race, keynesian_multiplier,        │
│                porter_five_forces                                 │
│    Priority 1: fearon_bargaining, wittman_zartman                 │
└──────────────────────────────────────────────────────────────────┘
```

---

1. Plugin Architecture

1.1 TheoryBase ABC (`core/theories/base.py`)

Every theory module is a class that:

1. Inherits from `TheoryBase`
2. Declares an inner `Parameters(BaseModel)` for Pydantic-validated params
3. Declares `state_variables` (reads / writes / initializes)
4. Implements `update(env, agents, tick) → dict[str, float]`

`update()` is a **pure function over the environment** — it reads from `env`, returns a delta dict, and never mutates `env` directly. The SimRunner accumulates all deltas and applies them once per tick, preventing ordering race conditions within the same priority bucket.

```python
# core/theories/base.py
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any

from pydantic import BaseModel

if TYPE_CHECKING:
    from core.agents.base import BDIAgent


@dataclass
class TheoryStateVariables:
    """
    Declares which environment keys this theory reads, writes, and initializes.
    SimRunner uses this at setup() to:
      - validate no two theories write the same key
      - seed the environment with initial values for keys this theory owns
    """
    reads: list[str] = field(default_factory=list)
    writes: list[str] = field(default_factory=list)
    initializes: list[str] = field(default_factory=list)


class TheoryBase(ABC):
    """
    Abstract base for all theory modules.

    Key conventions:
      - env keys owned by this theory: "{theory_id}__{variable_name}"
      - actor-specific keys:           "{actor_id}__{variable_name}"
      - cross-theory globals:          "global__{variable_name}"
      - all values normalized to [0, 1]
    """

    theory_id: str = "base"

    class Parameters(BaseModel):
        pass

    def __init__(self, parameters: dict[str, float] | None = None) -> None:
        self.params = self.Parameters(**(parameters or {}))
        self._tick_history: list[dict[str, float]] = []

    @property
    def state_variables(self) -> TheoryStateVariables:
        return TheoryStateVariables()

    def setup(self, env: dict[str, float]) -> dict[str, float]:
        """Called once before tick loop. Returns initial values to seed in env."""
        inits: dict[str, float] = {}
        for key in self.state_variables.initializes:
            if key not in env:
                inits[key] = 0.0
        return inits

    @abstractmethod
    def update(
        self,
        env: dict[str, float],
        agents: list["BDIAgent"],
        tick: int,
```

```
    ) -> dict[str, float]:
        """
        Core update. Called once per tick.
        Returns {env_key: new_value} — only keys in state_variables.writes.
        Do NOT mutate env directly.
        """
        ...

    def get_state_snapshot(self) -> dict[str, Any]:
        return {"theory_id": self.theory_id, "parameters": self.params.model_dump()}
```

1.2 Registry (`core/theories/__init__.py`)

```
from __future__ import annotations

from typing import Callable, Type

_THEORY_REGISTRY: dict[str, Type["TheoryBase"]] = {}


def register_theory(theory_id: str) -> Callable:
    """
    Class decorator. Registers the theory under `theory_id`.
    Auto-discovery pattern: importing the module registers it.
    SimRunner never needs a manual registry list.

    Usage:
        @register_theory("richardson_arms_race")
        class RichardsonArmsRace(TheoryBase):
            ...
    """
    def decorator(cls):
        cls.theory_id = theory_id
        _THEORY_REGISTRY[theory_id] = cls
        return cls
    return decorator


def get_theory(theory_id: str) -> Type["TheoryBase"]:
    if theory_id not in _THEORY_REGISTRY:
        raise KeyError(
            f"Theory '{theory_id}' not registered. "
            f"Available: {list(_THEORY_REGISTRY.keys())}"
        )
    return _THEORY_REGISTRY[theory_id]


def list_theories() -> list[str]:
    return sorted(_THEORY_REGISTRY.keys())
```

1.3 Normalization convention

All environment values are floats in **[0, 1]**. The mapping from real-world units to normalized space is theory-specific, documented per theory below. Clamp outputs: `max(0.0, min(1.0, value))`.

Tick unit is set on `SimSpec.tick_unit` and is one of `"month"`, `"quarter"`, `"year"`. Theories that implement differential equations scale their `dt` accordingly:

tick_unit	dt
month	1/12 ≈ 0.083

quarter	0.25
year	1.0

---

2. Theory 1 — Richardson Arms Race Model

2.1 Background

Lewis Fry Richardson (1960) derived a system of two coupled ordinary differential equations to describe the dynamics of mutual armament between two states. The model remains the canonical mathematical foundation for arms race analysis and appears in virtually every quantitative IR textbook.

The key insight: armament decisions are driven by three forces simultaneously — **reaction** to the adversary's arms (defense term), **economic drag** of maintaining arms (fatigue term), and **exogenous grievances** that motivate arming independent of the adversary (grievance term). Stability depends on whether the product of fatigue coefficients exceeds the product of defense coefficients.

2.2 Mathematical Formulation

```
dx/dt = k·y - a·x + g      (Country X's rate of arms change)
dy/dt = l·x - b·y + h      (Country Y's rate of arms change)

where:
  x, y  = arms level of Country X and Country Y
  k, l  = defense coefficients (reactivity to adversary)
  a, b  = fatigue coefficients (economic/political drag on own arms)
  g, h  = grievance terms (baseline arming independent of adversary)
```

Stability condition: `a·b > k·l`

If violated, the system diverges — arms race runaway escalation. This is the mathematically precise statement of an unstable arms race.

Equilibrium point (when `a·b > k·l`):

```
x* = (b·g + k·h) / (a·b - k·l)
y* = (a·h + l·g) / (a·b - k·l)
```

Escalation index (diagnostic aggregate):

```
escalation_index = (x + y) / 2
```

2.3 Parameter Table

Parameter	Description	Empirical Range	Cold War US-USSR	Default
`k`	X's defense coefficient (reactivity to Y's arms)	0.10 – 0.90	0.30 – 0.50	0.30
`l`	Y's defense coefficient (reactivity	0.10 – 0.90	0.30 – 0.50	0.30

	to X's arms)			
a	X's fatigue coefficient (drag on own arms)	0.05 – 0.40	0.15 – 0.25	0.15
b	Y's fatigue coefficient (drag on own arms)	0.05 – 0.40	0.15 – 0.25	0.15
g	X's grievance term (baseline arming motivation)	-0.10 – 0.30	0.05 – 0.10	0.05
h	Y's grievance term (baseline arming motivation)	-0.10 – 0.30	0.05 – 0.10	0.05
`tick_unit`	Time step scaling	—	—	"year"

Stability check at runtime: if `a·b ≤ k·l`, the simulation is in an unstable regime. Log a warning and clamp prevents overflow, but the trajectory is physically meaningful (runaway escalation until resource exhaustion at 1.0).

Negative grievance terms are valid: a state that is actively demilitarizing for domestic reasons has `g < 0`.

2.4 Normalization Mapping

Real-world concept	Env key	Normalization
Country X military readiness / arms level	`actor_a__military_readiness`	0 = disarmed, 1 = maximum military mobilization
Country Y military readiness / arms level	`actor_b__military_readiness`	0 = disarmed, 1 = maximum military mobilization
Combined escalation index	`richardson__escalation_index`	0 = both disarmed, 1 = both fully armed
Stability diagnostic	`richardson__stable`	1.0 if a·b > k·l, 0.0 otherwise

Note: `actor_a__military_readiness` and `actor_b__military_readiness` are **actor-namespaced** keys, not theory-namespaced. This means Fearon and Zartman theories can also read them without coupling to Richardson's namespace.

2.5 Python Class

```python
# core/theories/richardson_arms_race.py
"""
Richardson Arms Race Model (1960)

Differential equations:
  dx/dt = k·y - a·x + g
  dy/dt = l·x - b·y + h

Discretized per tick:
  x[t+1] = clamp(x[t] + dt · (k·y[t] - a·x[t] + g), 0, 1)
  y[t+1] = clamp(y[t] + dt · (l·x[t] - b·y[t] + h), 0, 1)

Stability: a·b > k·l  →  equilibrium exists
           a·b ≤ k·l  →  runaway escalation (unstable)
"""
from __future__ import annotations

import logging
from typing import TYPE_CHECKING
```

```python
from pydantic import BaseModel, Field, model_validator

from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables

if TYPE_CHECKING:
    from core.agents.base import BDIAgent

logger = logging.getLogger(__name__)

_DT_MAP = {"month": 1 / 12, "quarter": 0.25, "year": 1.0}


@register_theory("richardson_arms_race")
class RichardsonArmsRace(TheoryBase):
    """
    Lewis Fry Richardson (1960) mutual arms race dynamics.

    Domains: conflict, geopolitics, military
    Priority: 0 (runs before war-onset theories)
    """

    THEORY_ID = "richardson_arms_race"
    DOMAINS = ["conflict", "geopolitics", "military"]

    class Parameters(BaseModel):
        # defense coefficients
        k: float = Field(default=0.30, ge=0.0, le=1.0,
                         description="X's reactivity to Y's arms")
        l: float = Field(default=0.30, ge=0.0, le=1.0,
                         description="Y's reactivity to X's arms")
        # fatigue coefficients
        a: float = Field(default=0.15, ge=0.0, le=1.0,
                         description="X's economic drag on own arms")
        b: float = Field(default=0.15, ge=0.0, le=1.0,
                         description="Y's economic drag on own arms")
        # grievance terms
        g: float = Field(default=0.05, ge=-0.5, le=0.5,
                         description="X's baseline arming motivation")
        h: float = Field(default=0.05, ge=-0.5, le=0.5,
                         description="Y's baseline arming motivation")
        # simulation config
        tick_unit: str = Field(default="year",
                               description="'month', 'quarter', or 'year'")
        actor_a_id: str = Field(default="actor_a",
                                description="env key prefix for actor A")
        actor_b_id: str = Field(default="actor_b",
                                description="env key prefix for actor B")

        @model_validator(mode="after")
        def warn_if_unstable(self) -> "RichardsonArmsRace.Parameters":
            if self.a * self.b <= self.k * self.l:
                logger.warning(
                    "Richardson parameters are in UNSTABLE regime: "
                    "a·b=%.4f ≤ k·l=%.4f. Expect runaway escalation.",
                    self.a * self.b, self.k * self.l
                )
            return self

    @property
    def state_variables(self) -> TheoryStateVariables:
        p = self.params
        return TheoryStateVariables(
            reads=[
                f"{p.actor_a_id}__military_readiness",
                f"{p.actor_b_id}__military_readiness",
            ],
            writes=[
                f"{p.actor_a_id}__military_readiness",
```

```
                f"{p.actor_b_id}__military_readiness",
                "richardson__escalation_index",
                "richardson__stable",
            ],
            initializes=[
                f"{p.actor_a_id}__military_readiness",
                f"{p.actor_b_id}__military_readiness",
                "richardson__escalation_index",
                "richardson__stable",
            ],
        )

    def update(
        self,
        env: dict[str, float],
        agents: list["BDIAgent"],
        tick: int,
    ) -> dict[str, float]:
        p = self.params
        dt = _DT_MAP.get(p.tick_unit, 1.0)

        # dx/dt = k·y - a·x + g
        x = env.get(f"{p.actor_a_id}__military_readiness", 0.5)
        y = env.get(f"{p.actor_b_id}__military_readiness", 0.5)

        dx = p.k * y - p.a * x + p.g
        dy = p.l * x - p.b * y + p.h

        new_x = max(0.0, min(1.0, x + dt * dx))
        new_y = max(0.0, min(1.0, y + dt * dy))

        # Stability diagnostic: 1.0 = stable equilibrium exists
        stable = 1.0 if (p.a * p.b > p.k * p.l) else 0.0

        return {
            f"{p.actor_a_id}__military_readiness": new_x,
            f"{p.actor_b_id}__military_readiness": new_y,
            "richardson__escalation_index": (new_x + new_y) / 2.0,
            "richardson__stable": stable,
        }
```

2.6 Environment Variable Reference

Key	Type	Range	Description
`actor_a__military_readiness`	write	[0, 1]	Arms level of actor A; 0 = disarmed, 1 = fully mobilized
`actor_b__military_readiness`	write	[0, 1]	Arms level of actor B
`richardson__escalation_index`	write	[0, 1]	(x + y) / 2; overall escalation barometer
`richardson__stable`	write	{0, 1}	1.0 if a·b > k·l; 0.0 if system is in runaway regime

2.7 Key Limitations

- **Two-actor only.** The classic Richardson model is bilateral. Extending to N actors requires a full NxN defense coefficient matrix; not yet implemented.

- **Linear dynamics.** Real arms races involve threshold effects (e.g., nuclear deterrence stability after mutual second-strike capability). Nonlinear extensions exist (e.g., Intriligator-Brito) but are a separate theory module.
- **No resource ceiling.** The model is bounded only by the [0,1] clamp, not by an explicit GDP budget. For fiscal realism, couple with `keynesian_multiplier` and read `keynesian__gdp_normalized` to scale `a` dynamically.
- **Parameter identification.** k and l are empirically estimated from observed arms spending series. In Crucible, Forge populates them from research context; default values are appropriate for two middle-power states.

---

3. Theory 2 — Fearon Bargaining Model of War

3.1 Background

James Fearon (1995), "Rationalist Explanations for War," *International Organization* 49(3): 379–414. One of the most-cited papers in international relations.

Fearon's central question: why do wars occur when they are costly for both sides, and a negotiated settlement that both parties prefer *ex ante* always exists? His answer: **private information** (each side's true military capability is private, and incentives to bluff prevent credible revelation) and **commitment problems** (a state that expects its power to decline cannot credibly promise not to exploit future superiority).

The model provides a clean decision rule: war occurs when the private information gap exceeds the combined costs of fighting, OR when the rate of relative power shift is too large for commitment.

3.2 Mathematical Formulation

```
Notation:
  p      = Actor A's true probability of winning conflict (private to A)
  p̂_B    = Actor B's estimate of A's win probability (B's belief)
  V      = value of contested prize (normalized to 1.0 in Crucible)
  c_A    = A's cost of fighting (fraction of V; e.g., 0.1 = 10% of prize)
  c_B    = B's cost of fighting (fraction of V)

Settlement range:
  Any split s in [p - c_A,  p + c_B] is preferred by BOTH sides
  over fighting.
  Width of settlement range = c_A + c_B

War occurs (private information mechanism) when:
  |p - p̂_B| > c_A + c_B
  i.e., B's misestimate of A's strength exceeds the joint cost cushion.

War occurs (commitment problem mechanism) when:
  power_shift_rate > commitment_threshold
  where power_shift_rate = |p[t] - p[t-1]| / dt

Conflict probability (combined):
  P(conflict) = f(info_gap, settlement_range_width, power_shift_rate)

  A tractable linear specification used in Crucible:
    info_gap  = |p - p̂_B|
    info_term = max(0, info_gap - (c_A + c_B)) / (1 - (c_A + c_B))
```

```
    commit_term = max(0, power_shift_rate - commit_threshold) / (1 - commit_threshold)
    P(conflict)[t] = clamp(max(info_term, commit_term), 0, 1)
```

3.3 Parameter Table

Parameter	Description	Empirical Range	Default
`c_A`	A's cost of war (fraction of prize)	0.05 – 0.20	0.10
`c_B`	B's cost of war (fraction of prize)	0.05 – 0.20	0.10
`private_info_sigma`	Std dev of B's estimation error about p	0.10 – 0.40	0.20
`commitment_threshold`	Power shift rate above which commitment fails	0.03 – 0.10	0.05
`tick_unit`	Time scaling	—	"year"

Fearon's empirical calibration: war costs in most interstate conflicts represent 5–20% of the contested prize (resources, territory, regime survival). The private information parameter is harder to calibrate directly; 0.15–0.25 is consistent with post-Cold War crisis data.

3.4 Normalization Mapping

Real-world concept	Env key	Normalization
A's true win probability	`fearon__win_prob_a`	[0, 1] = A's probability of prevailing in conflict
B's estimate of A's win prob	`fearon__win_prob_b_estimate`	[0, 1] = B's belief about A's capability
A's war cost	`fearon__war_cost_a`	[0, 1] = cost as fraction of prize value
B's war cost	`fearon__war_cost_b`	[0, 1] = cost as fraction of prize value
Conflict probability	`fearon__conflict_probability`	[0, 1] = per-tick probability of war onset
Settlement range width	`fearon__settlement_range_width`	[0, 1] = c_A + c_B

`fearon__win_prob_a` is written by Fearon theory but also read by Richardson (escalation affects win probability) and Zartman (ripeness depends on military balance). This is the primary cross-theory bridge variable.

3.5 Python Class

```
# core/theories/fearon_bargaining.py
"""
Fearon Bargaining Model of War (1995)

War occurs when:
  (a) private information gap > joint war costs:
        |p - p̂_B| > c_A + c_B
  (b) power shift rate exceeds commitment threshold:
        |p[t] - p[t-1]| / dt > commit_threshold

Conflict probability is a smooth [0,1] output derived from both mechanisms.
```

```python
Reference: Fearon (1995), International Organization 49(3): 379–414
"""
from __future__ import annotations

import logging
from typing import TYPE_CHECKING

from pydantic import BaseModel, Field

from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables

if TYPE_CHECKING:
    from core.agents.base import BDIAgent

logger = logging.getLogger(__name__)

_DT_MAP = {"month": 1 / 12, "quarter": 0.25, "year": 1.0}


@register_theory("fearon_bargaining")
class FearonBargaining(TheoryBase):
    """
    Rationalist bargaining theory of war onset.

    Domains: conflict, geopolitics, crisis management
    Priority: 1 (reads military readiness set by Richardson at priority 0)
    Conflicts with: wittman_zartman (competing war-termination models)
    """

    THEORY_ID = "fearon_bargaining"
    DOMAINS = ["conflict", "geopolitics", "crisis_management"]

    class Parameters(BaseModel):
        c_A: float = Field(default=0.10, ge=0.0, le=1.0,
                           description="A's war cost as fraction of prize")
        c_B: float = Field(default=0.10, ge=0.0, le=1.0,
                           description="B's war cost as fraction of prize")
        private_info_sigma: float = Field(
            default=0.20, ge=0.0, le=1.0,
            description="Std dev of B's estimation error re: A's win prob"
        )
        commitment_threshold: float = Field(
            default=0.05, ge=0.0, le=1.0,
            description="Power shift rate above which commitment problem activates"
        )
        tick_unit: str = Field(default="year")
        actor_a_id: str = Field(default="actor_a")
        actor_b_id: str = Field(default="actor_b")

    @property
    def state_variables(self) -> TheoryStateVariables:
        return TheoryStateVariables(
            reads=[
                "fearon__win_prob_a",
                "fearon__win_prob_b_estimate",
                "fearon__war_cost_a",
                "fearon__war_cost_b",
                # cross-theory: read military readiness from Richardson
                "actor_a__military_readiness",
                "actor_b__military_readiness",
            ],
            writes=[
                "fearon__win_prob_a",
                "fearon__win_prob_b_estimate",
                "fearon__war_cost_a",
                "fearon__war_cost_b",
                "fearon__conflict_probability",
                "fearon__settlement_range_width",
            ],
```

```python
        initializes=[
            "fearon__win_prob_a",
            "fearon__win_prob_b_estimate",
            "fearon__war_cost_a",
            "fearon__war_cost_b",
            "fearon__conflict_probability",
            "fearon__settlement_range_width",
        ],
    )

def update(
    self,
    env: dict[str, float],
    agents: list["BDIAgent"],
    tick: int,
) -> dict[str, float]:
    p = self.params

    # Win probability: derive from military readiness if available,
    # else use stored estimate. mil_a / (mil_a + mil_b) is the
    # Lanchester-flavored capability ratio.
    mil_a = env.get(f"{p.actor_a_id}__military_readiness", 0.5)
    mil_b = env.get(f"{p.actor_b_id}__military_readiness", 0.5)
    denom = mil_a + mil_b
    win_prob_a = (mil_a / denom) if denom > 1e-6 else 0.5

    # B's estimate = true prob + Gaussian noise drawn from prior
    # In a tick model we evolve the estimate with partial update:
    # p̂_B moves 10% toward true value per tick (slow belief update)
    prev_estimate = env.get("fearon__win_prob_b_estimate", 0.5)
    belief_update_rate = 0.10
    win_prob_b_estimate = prev_estimate + belief_update_rate * (win_prob_a - prev_estimate)

    # War costs: read from env (agents or shocks may have updated them),
    # fall back to calibrated params
    war_cost_a = env.get("fearon__war_cost_a", p.c_A)
    war_cost_b = env.get("fearon__war_cost_b", p.c_B)

    settlement_range_width = war_cost_a + war_cost_b

    # Private information mechanism:
    # War if |p - p̂_B| > c_A + c_B
    info_gap = abs(win_prob_a - win_prob_b_estimate)
    excess_info_gap = max(0.0, info_gap - settlement_range_width)
    remaining_range = max(1e-6, 1.0 - settlement_range_width)
    info_conflict_term = excess_info_gap / remaining_range

    # Commitment problem mechanism:
    # War if relative power shift rate exceeds threshold
    prev_win_prob = env.get("fearon__win_prob_a", 0.5)
    dt = _DT_MAP.get(p.tick_unit, 1.0)
    power_shift_rate = abs(win_prob_a - prev_win_prob) / max(dt, 1e-6)
    excess_shift = max(0.0, power_shift_rate - p.commitment_threshold)
    remaining_shift = max(1e-6, 1.0 - p.commitment_threshold)
    commit_conflict_term = min(1.0, excess_shift / remaining_shift)

    # Combined conflict probability: max of both mechanisms
    conflict_probability = max(0.0, min(1.0,
        max(info_conflict_term, commit_conflict_term)
    ))

    return {
        "fearon__win_prob_a": win_prob_a,
        "fearon__win_prob_b_estimate": win_prob_b_estimate,
        "fearon__war_cost_a": war_cost_a,
        "fearon__war_cost_b": war_cost_b,
        "fearon__conflict_probability": conflict_probability,
        "fearon__settlement_range_width": settlement_range_width,
    }
```

3.6 Environment Variable Reference

Key	Type	Range	Description
`fearon__win_prob_a`	write	[0, 1]	A's true probability of prevailing in conflict
`fearon__win_prob_b_estimate`	write	[0, 1]	B's (lagged) estimate of A's win probability
`fearon__war_cost_a`	read/write	[0, 1]	A's cost of war as fraction of contested prize
`fearon__war_cost_b`	read/write	[0, 1]	B's cost of war
`fearon__conflict_probability`	write	[0, 1]	Per-tick probability of war onset
`fearon__settlement_range_width`	write	[0, 1]	Width of the negotiable settlement zone (c_A + c_B)

3.7 Key Limitations

- **Binary war/peace.** Fearon's model predicts onset, not intensity or duration. Once conflict begins, the theory has no endogenous termination mechanism — use Zartman for that.
- **Conflict with Zartman.** `fearon_bargaining` and `wittman_zartman` are registered as `THEORY_CONFLICTS` in `ARCHITECTURE-FORGE.md`. Only one should run per simulation. Fearon is preferred for onset-focused scenarios; Zartman for termination/mediation scenarios.
- **Deterministic belief update.** The 10% per-tick belief convergence rate is a simplification. A proper implementation would sample from a Beta distribution updated via Bayes' rule.
- **No repeated interaction.** Fearon (1995) is a one-shot game. Reputation and learning effects from prior interactions require a repeated-game extension.

---

4. Theory 3 — Wittman-Zartman Settlement / Ripeness Theory

4.1 Background

Two complementary theories are unified in this module:

- **Wittman (1979)**, "How Wars End," *Journal of Conflict Resolution* 23(4): 743–763: expected utility framework for war termination. A party prefers settlement when the expected utility of settling exceeds the expected utility of continued fighting.
- **Zartman (1985)**, "Ripe for Resolution": the **Mutually Hurting Stalemate** (MHS) — the empirical observation that conflicts become negotiable when both parties simultaneously experience costs exceeding the value of continued fighting AND when neither side sees a path to decisive victory. MHS is the "ripe moment" that makes mediation effective.

Together they answer: not just *when* war ends, but *when negotiations have a realistic chance of success*.

4.2 Mathematical Formulation

```
Expected utility of continuing conflict (fighting):
  EU_war(A) = p · V - c_A
  EU_war(B) = (1-p) · V - c_B

  where p = A's win probability, V = prize value (normalized = 1.0)

Expected utility of settlement:
  EU_settle(A) = s_A - transaction_costs
  EU_settle(B) = s_B - transaction_costs
  s_A + s_B = 1  (settlement splits the prize)
  transaction_costs ∈ [0, 0.10]  (negotiation overhead)

A prefers settlement when:
  s_A > EU_war(A) + transaction_costs
  i.e., s_A > p·V - c_A + transaction_costs

Mutually Hurting Stalemate (MHS):
  MHS = (EU_war(A) < payoff_floor_A) AND (EU_war(B) < payoff_floor_B)

  payoff_floor = current payoff from neither winning nor losing
                 (status quo, normalized to 0.5 · V = 0.5 in Crucible)

Ripe moment:
  ripe = MHS
         AND stalemate_duration ≥ min_stalemate_ticks
         AND (mediator_present OR urgency_factor > urgency_threshold)

Negotiation probability per tick:
  P(negotiate | ripe) = base_rate · ripe_multiplier · (1 + urgency_factor)
  P(negotiate | not ripe) = base_rate

  base_rate ≈ 0.02–0.05/tick (year)
  ripe_multiplier ≈ 3–8x empirically (Zartman 1985, Touval & Zartman 1985)
```

4.3 Parameter Table

Parameter	Description	Empirical Range	Default
`base_negotiation_rate`	P(negotiate) per tick in non-ripe conditions	0.01 – 0.05	0.02
`ripe_multiplier`	Multiplier on negotiation probability when ripe	3.0 – 8.0	5.0
`min_stalemate_ticks`	Minimum stalemate duration before ripe condition	2 – 8 ticks	4
`payoff_floor`	Threshold below which EU_war triggers MHS	0.3 – 0.5	0.40
`transaction_costs`	Overhead cost of negotiation itself	0.01 – 0.10	0.05
`urgency_threshold`	Urgency level that substitutes for mediator	0.5 – 0.8	0.65

4.4 Normalization Mapping

Real-world concept	Env key	Normalization
A's EU of continued war	`zartman__eu_war_a`	[0, 1] = p·V - c_A, scaled to [0,1]
B's EU of continued war	`zartman__eu_war_b`	[0, 1] = (1-p)·V - c_B, scaled to [0,1]
Mutually hurting stalemate flag	`zartman__mhs`	1.0 = MHS active, 0.0 = not
Ripe moment flag	`zartman__ripe_moment`	1.0 = ripe, 0.0 = not ripe
Negotiation probability	`zartman__negotiation_probability`	[0, 1] = per-tick P(negotiation begins)
Stalemate duration	`zartman__stalemate_duration`	[0, 1] = ticks_in_stalemate / max_stalemate_horizon
Mediator present	`zartman__mediator_present`	1.0 = active mediator, 0.0 = none

4.5 Python Class

```python
# core/theories/wittman_zartman.py
"""
Wittman (1979) / Zartman (1985) Ripeness Theory

War termination expected utility:
  EU_war(A) = p · V - c_A
  EU_war(B) = (1-p) · V - c_B

MHS (Mutually Hurting Stalemate):
  Both parties have EU_war < payoff_floor AND no path to decisive victory.

Ripe moment:
  ripe = MHS AND stalemate_duration >= min_stalemate_ticks
         AND (mediator_present OR urgency_factor > urgency_threshold)

Negotiation probability:
  base_rate · ripe_multiplier · (1 + urgency) if ripe, else base_rate

References:
  Wittman (1979) Journal of Conflict Resolution 23(4)
  Zartman (1985) Ripe for Resolution
"""
from __future__ import annotations

import logging
from typing import TYPE_CHECKING

from pydantic import BaseModel, Field

from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables

if TYPE_CHECKING:
    from core.agents.base import BDIAgent

logger = logging.getLogger(__name__)

_MAX_STALEMATE_HORIZON = 48  # ticks (e.g., 48 months = 4 years)


@register_theory("wittman_zartman")
class WittmanZartman(TheoryBase):
    """
    War termination and ripeness theory.
```

```python
Domains: conflict, geopolitics, mediation
Priority: 1 (reads Richardson/Fearon outputs from priority 0)
Conflicts with: fearon_bargaining (select one per sim)
"""

THEORY_ID = "wittman_zartman"
DOMAINS = ["conflict", "geopolitics", "mediation"]

class Parameters(BaseModel):
    base_negotiation_rate: float = Field(default=0.02, ge=0.0, le=1.0)
    ripe_multiplier: float = Field(default=5.0, ge=1.0, le=20.0)
    min_stalemate_ticks: int = Field(default=4, ge=1)
    payoff_floor: float = Field(default=0.40, ge=0.0, le=1.0,
                                description="EU_war below this → MHS for that party")
    transaction_costs: float = Field(default=0.05, ge=0.0, le=0.5)
    urgency_threshold: float = Field(default=0.65, ge=0.0, le=1.0)
    actor_a_id: str = Field(default="actor_a")
    actor_b_id: str = Field(default="actor_b")

def __init__(self, parameters: dict[str, float] | None = None) -> None:
    super().__init__(parameters)
    self._stalemate_ticks: int = 0  # internal counter, not in env

@property
def state_variables(self) -> TheoryStateVariables:
    return TheoryStateVariables(
        reads=[
            "fearon__win_prob_a",
            "fearon__war_cost_a",
            "fearon__war_cost_b",
            "zartman__mediator_present",
            "global__urgency_factor",
        ],
        writes=[
            "zartman__eu_war_a",
            "zartman__eu_war_b",
            "zartman__mhs",
            "zartman__ripe_moment",
            "zartman__negotiation_probability",
            "zartman__stalemate_duration",
        ],
        initializes=[
            "zartman__eu_war_a",
            "zartman__eu_war_b",
            "zartman__mhs",
            "zartman__ripe_moment",
            "zartman__negotiation_probability",
            "zartman__stalemate_duration",
            "zartman__mediator_present",
            "global__urgency_factor",
        ],
    )

def update(
    self,
    env: dict[str, float],
    agents: list["BDIAgent"],
    tick: int,
) -> dict[str, float]:
    p = self.params

    # Read military balance and costs
    win_prob_a = env.get("fearon__win_prob_a", 0.5)
    c_A = env.get("fearon__war_cost_a", 0.10)
    c_B = env.get("fearon__war_cost_b", 0.10)

    # EU_war(A) = p · V - c_A  (V normalized to 1.0)
    # EU_war(B) = (1-p) · V - c_B
    eu_war_a = max(0.0, min(1.0, win_prob_a - c_A))
    eu_war_b = max(0.0, min(1.0, (1.0 - win_prob_a) - c_B))
```

```
        # MHS: both parties below payoff floor
        mhs_a = eu_war_a < p.payoff_floor
        mhs_b = eu_war_b < p.payoff_floor
        mhs = 1.0 if (mhs_a and mhs_b) else 0.0

        # Stalemate duration counter
        if mhs_a and mhs_b:
            self._stalemate_ticks += 1
        else:
            self._stalemate_ticks = 0
        stalemate_duration_norm = min(1.0,
            self._stalemate_ticks / _MAX_STALEMATE_HORIZON
        )

        # Ripeness
        mediator_present = env.get("zartman__mediator_present", 0.0) > 0.5
        urgency = env.get("global__urgency_factor", 0.0)
        urgency_high = urgency > p.urgency_threshold

        duration_met = self._stalemate_ticks >= p.min_stalemate_ticks
        ripe = (
            mhs_a and mhs_b
            and duration_met
            and (mediator_present or urgency_high)
        )
        ripe_val = 1.0 if ripe else 0.0

        # Negotiation probability
        if ripe:
            negotiation_prob = min(1.0,
                p.base_negotiation_rate * p.ripe_multiplier * (1.0 + urgency)
            )
        else:
            negotiation_prob = p.base_negotiation_rate

        return {
            "zartman__eu_war_a": eu_war_a,
            "zartman__eu_war_b": eu_war_b,
            "zartman__mhs": mhs,
            "zartman__ripe_moment": ripe_val,
            "zartman__negotiation_probability": negotiation_prob,
            "zartman__stalemate_duration": stalemate_duration_norm,
        }
```

4.6 Environment Variable Reference

Key	Type	Range	Description
`zartman__eu_war_a`	write	[0, 1]	A's expected utility from continued fighting
`zartman__eu_war_b`	write	[0, 1]	B's expected utility from continued fighting
`zartman__mhs`	write	{0, 1}	1.0 = Mutually Hurting Stalemate active
`zartman__ripe_moment`	write	{0, 1}	1.0 = all ripeness conditions met
`zartman__negotiation_probability`	write	[0, 1]	Per-tick probability of negotiation onset
`zartman__stalemate_duration`	write	[0, 1]	Normalized stalemate length (0–48 ticks)

`zartman__mediator_present`	read	{0, 1}	Set by external shock or agent action
`global__urgency_factor`	read	[0, 1]	Cross-theory urgency signal

4.7 Key Limitations

- **Binary ripe/not-ripe.** Ripeness in Zartman is conceptually a threshold, but empirically it is a continuum. The current implementation uses hard conditions; a probabilistic version would use a logistic function over the ripeness indicators.
- **Internal stalemate counter.** `_stalemate_ticks` lives on the theory instance, not in env. This means it is not captured in env snapshots. A future version should expose it as `zartman__stalemate_ticks_raw` (integer stored as float).
- **No endogenous mediator.** The mediator is an exogenous input (`zartman__mediator_present` set by shock). A richer model would have a mediator agent whose BDI goals include entering when MHS ≥ threshold.
- **Conflict with Fearon.** Both models output a negotiation/conflict probability. Running both simultaneously produces double-counting. `THEORY_CONFLICTS` in Forge enforces mutual exclusion.

---

5. Theory 4 — Keynesian Multiplier / Fiscal Shock Propagation

5.1 Background

The Keynesian multiplier (Keynes 1936; empirically characterized by Blanchard & Perotti 2002, Ramey 2011) describes how a fiscal shock — government spending increase, tax cut, trade disruption, or sanctions — propagates through the economy via consumption rounds. Each dollar of initial spending generates additional spending as recipients spend their income, with the total effect determined by the multiplier M.

In Crucible, this theory serves three primary use cases:

1. **Sanctions modeling**: a trade shock reduces export revenue, which propagates through the domestic economy with multiplier M.
2. **Fiscal stimulus/austerity**: a government increases or cuts spending; M determines the GDP trajectory.
3. **Military spending spillover**: arms buildups (Richardson theory) create fiscal shocks that feed back into economic capacity.

5.2 Mathematical Formulation

```
Keynesian multiplier:
  M = 1 / (1 - MPC·(1-t) + m)

  where:
    MPC = marginal propensity to consume (fraction of income spent on consumption)
    t   = effective tax rate (fraction of income collected as taxes)
    m   = marginal propensity to import (fraction of income spent on imports)

GDP impact of a fiscal shock (applied with lag):
  GDP_impact = M · fiscal_shock

Distributed lag (geometric decay, applied over lag_ticks):
  impact_fraction[lag] = (1 - decay_rate)^lag · decay_rate
  GDP_impact[t + lag] += impact_fraction[lag] · M · fiscal_shock
```

```
Okun's Law (GDP growth → unemployment change):
  Δunemployment ≈ -0.5 · ΔGDP_growth
  (Okun's coefficient empirically -0.4 to -0.6; use -0.5 as baseline)

For sanctions modeling:
  fiscal_shock = -trade_revenue_loss · exposure_coefficient
  trade_revenue_loss = trade_volume_lost · (export_price_normalized)
  exposure_coefficient ∈ [0.5, 1.5] (sector concentration effect)
```

Multiplier ranges by economy type:

Economy type	MPC	t	m	Typical M
Large developed (US, EU)	0.70–0.80	0.25–0.35	0.10–0.20	1.5–2.5
Small open (Netherlands, Singapore)	0.60–0.75	0.30–0.40	0.35–0.50	0.8–1.3
Developing economy	0.75–0.90	0.15–0.25	0.15–0.35	1.1–1.8
Sanctions target (Russia 2022 analog)	0.65–0.75	0.20–0.30	0.15–0.25	1.2–1.6

5.3 Parameter Table

Parameter	Description	Empirical Range	Default
`mpc`	Marginal propensity to consume	0.50 – 0.90	0.72
`tax_rate`	Effective tax rate	0.15 – 0.45	0.28
`import_propensity`	Marginal propensity to import	0.10 – 0.50	0.18
`lag_ticks`	Multiplier propagation lag in ticks	1 – 8	4
`decay_rate`	Geometric decay rate of pending impact	0.20 – 0.60	0.35
`okun_coefficient`	Unemployment response to GDP change	-0.60 – -0.40	-0.50
`sanctions_exposure`	Amplifier for sanctions channel	0.50 – 1.50	1.0

5.4 Normalization Mapping

Real-world concept	Env key	Normalization
Normalized GDP level	`keynesian__gdp_normalized`	0.5 = baseline; [0,1] span ≈ ±40% of baseline
Pending fiscal shock	`keynesian__fiscal_shock_pending`	[-1, 1] as signed float; positive = stimulus, negative = contraction
Current multiplier value	`keynesian__multiplier`	[0, 1] mapped from M ∈ [0, 3]; M/3

Unemployment rate	`keynesian__unemployment`	[0, 1]; 0.05 = structural minimum, 1.0 = extreme
Marginal propensity to consume	`keynesian__mpc`	Direct [0,1]
Global trade volume	`global__trade_volume`	[0, 1]; 0.5 = baseline world trade

Note: `keynesian__fiscal_shock_pending` is a signed float that technically violates the [0,1] convention. The env stores it as `0.5 + shock/2` (center at 0.5, ±0.5 range) and the theory decodes it as `(raw - 0.5) * 2`. This preserves the [0,1] container while representing signed shocks.

5.5 Python Class

```python
# core/theories/keynesian_multiplier.py
"""
Keynesian Multiplier / Fiscal Shock Propagation

Multiplier formula:
  M = 1 / (1 - MPC·(1-t) + m)

GDP dynamics (distributed lag geometric decay):
  pending_shock accumulates, releases at rate decay_rate per tick
  GDP_delta[t] = pending_shock[t] · decay_rate · M

Okun's Law:
  Δunemployment ≈ okun_coefficient · ΔGDP_growth

Sanctions channel:
  fiscal_shock = -trade_revenue_loss · sanctions_exposure
  encoded as (0.5 + shock/2) in env to maintain [0,1] convention.

References:
  Keynes (1936); Blanchard & Perotti (2002) QJE; Ramey (2011) JEL
"""
from __future__ import annotations

import logging
from typing import TYPE_CHECKING

from pydantic import BaseModel, Field

from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables

if TYPE_CHECKING:
    from core.agents.base import BDIAgent

logger = logging.getLogger(__name__)

# GDP normalization: 0.5 = baseline, range maps ±40% real GDP deviation
_GDP_BASELINE = 0.5
_GDP_SCALE = 0.40  # 1.0 normalized = 40% above baseline real GDP

# Fiscal shock encoding: store as (0.5 + shock/2)
def _encode_shock(shock: float) -> float:
    return max(0.0, min(1.0, 0.5 + shock / 2.0))

def _decode_shock(encoded: float) -> float:
    return (encoded - 0.5) * 2.0


@register_theory("keynesian_multiplier")
class KeynesianMultiplier(TheoryBase):
    """
    Keynesian multiplier and fiscal shock propagation.
```

```python
Domains: macro, sanctions, economics
Priority: 0 (runs before market theories)
"""

THEORY_ID = "keynesian_multiplier"
DOMAINS = ["macro", "sanctions", "economics"]

class Parameters(BaseModel):
    mpc: float = Field(default=0.72, ge=0.0, le=1.0,
                       description="Marginal propensity to consume")
    tax_rate: float = Field(default=0.28, ge=0.0, le=1.0,
                            description="Effective tax rate")
    import_propensity: float = Field(default=0.18, ge=0.0, le=1.0,
                                     description="Marginal propensity to import")
    lag_ticks: int = Field(default=4, ge=1,
                           description="Propagation lag in ticks")
    decay_rate: float = Field(default=0.35, ge=0.01, le=1.0,
                              description="Geometric decay of pending shock per tick")
    okun_coefficient: float = Field(default=-0.50, ge=-1.0, le=0.0,
                                    description="Unemployment response to GDP change")
    sanctions_exposure: float = Field(default=1.0, ge=0.0, le=2.0,
                                      description="Amplifier for trade shock channel")

@property
def state_variables(self) -> TheoryStateVariables:
    return TheoryStateVariables(
        reads=[
            "keynesian__fiscal_shock_pending",
            "keynesian__gdp_normalized",
            "keynesian__unemployment",
            "global__trade_volume",
        ],
        writes=[
            "keynesian__gdp_normalized",
            "keynesian__fiscal_shock_pending",
            "keynesian__multiplier",
            "keynesian__unemployment",
            "keynesian__mpc",
        ],
        initializes=[
            "keynesian__gdp_normalized",
            "keynesian__fiscal_shock_pending",
            "keynesian__multiplier",
            "keynesian__unemployment",
            "keynesian__mpc",
            "global__trade_volume",
        ],
    )

def setup(self, env: dict[str, float]) -> dict[str, float]:
    inits = super().setup(env)
    # Override defaults: GDP starts at baseline, unemployment at structural minimum
    if "keynesian__gdp_normalized" not in env:
        inits["keynesian__gdp_normalized"] = 0.5
    if "keynesian__unemployment" not in env:
        inits["keynesian__unemployment"] = 0.05  # 5% structural unemployment
    if "keynesian__fiscal_shock_pending" not in env:
        inits["keynesian__fiscal_shock_pending"] = 0.5  # encoded zero shock
    if "global__trade_volume" not in env:
        inits["global__trade_volume"] = 0.5
    return inits

def update(
    self,
    env: dict[str, float],
    agents: list["BDIAgent"],
    tick: int,
) -> dict[str, float]:
    p = self.params
```

```
        # Compute multiplier: M = 1 / (1 - MPC*(1-t) + m)
        denominator = 1.0 - p.mpc * (1.0 - p.tax_rate) + p.import_propensity
        # Guard against degenerate parameters (denominator → 0)
        denominator = max(denominator, 0.05)
        M = 1.0 / denominator
        # Normalize M to [0,1]: M typically 0.8–2.5, cap at 3.0
        multiplier_norm = min(1.0, M / 3.0)

        # Read pending shock (decode from [0,1] container)
        shock_encoded = env.get("keynesian__fiscal_shock_pending", 0.5)
        pending_shock = _decode_shock(shock_encoded)

        # Trade shock channel (sanctions): if trade volume drops below baseline,
        # generate a negative fiscal shock
        trade_volume = env.get("global__trade_volume", 0.5)
        trade_baseline = 0.5
        if trade_volume < trade_baseline:
            trade_shock = (trade_volume - trade_baseline) * p.sanctions_exposure
            pending_shock += trade_shock

        # Release this tick's portion of pending shock
        released_shock = pending_shock * p.decay_rate
        remaining_shock = pending_shock * (1.0 - p.decay_rate)

        # GDP impact = released_shock * M (scaled by GDP_SCALE)
        gdp_current = env.get("keynesian__gdp_normalized", 0.5)
        gdp_delta = released_shock * M * _GDP_SCALE
        gdp_new = max(0.0, min(1.0, gdp_current + gdp_delta))

        # Okun's Law: Δunemployment ≈ okun_coefficient · ΔGDP_growth
        gdp_growth = gdp_new - gdp_current
        unemployment_current = env.get("keynesian__unemployment", 0.05)
        unemployment_delta = p.okun_coefficient * gdp_growth * 2.0  # *2 to scale to unemployment units
        unemployment_new = max(0.0, min(1.0, unemployment_current + unemployment_delta))

        return {
            "keynesian__gdp_normalized": gdp_new,
            "keynesian__fiscal_shock_pending": _encode_shock(remaining_shock),
            "keynesian__multiplier": multiplier_norm,
            "keynesian__unemployment": unemployment_new,
            "keynesian__mpc": p.mpc,
        }
```

5.6 Environment Variable Reference

Key	Type	Range	Description
`keynesian__gdp_normalized`	write	[0, 1]	GDP level; 0.5 = baseline, 0 ≈ -40%, 1 ≈ +40%
`keynesian__fiscal_shock_pending`	read/write	[0, 1]	Encoded pending shock; decode as (val - 0.5) * 2
`keynesian__multiplier`	write	[0, 1]	M / 3.0; actual multiplier value = val * 3
`keynesian__unemployment`	write	[0, 1]	Unemployment rate; 0.05 = structural floor
`keynesian__mpc`	write	[0, 1]	MPC parameter (direct passthrough for observability)
`global__trade_volume`	read	[0, 1]	Trade volume index; 0.5 = baseline

5.7 Key Limitations

- **Signed shock encoding.** The `(val - 0.5) * 2` convention is a workaround for the [0,1] normalization contract. Any theory or agent that injects a fiscal shock must use `_encode_shock()`. This is a known friction point; a future version may add a `signed_env` separate dict for flows.
- **Static multiplier.** In reality M changes with the business cycle (fiscal multipliers are higher in recessions). The current implementation uses fixed parameters; a dynamic M would read `keynesian__gdp_normalized` and scale MPC.
- **No explicit lag queue.** The geometric decay approximates a distributed lag but does not model the actual lag structure (e.g., spending hits in tick 2, peak effect in tick 3–5). A proper implementation would maintain a `_pending_by_lag: list[float]` queue on the instance.
- **Okun's Law scaling.** The `* 2.0` scaling factor is a rough calibration. Proper calibration requires knowing the GDP scale (±40%) relative to unemployment scale (0–100%). Revisit if scenario-specific calibration is needed.

---

6. Theory 5 — Porter's Five Forces

6.1 Background

Michael Porter (1980), *Competitive Strategy: Techniques for Analyzing Industries and Competitors*. Porter's Five Forces is the standard framework for analyzing industry-level competitive dynamics and the structural determinants of profitability. In Crucible, it enables corporate strategy scenarios — market entry decisions, competitive response modeling, and industry structure evolution over time.

Each of the five forces is a normalized [0,1] variable. The forces evolve each tick based on actor actions (e.g., investment in capacity raises barriers to entry; M&A changes buyer/supplier concentration; R&D creates or erodes substitute threats). Profitability is a weighted combination of the forces.

6.2 Mathematical Formulation

```
Five force variables (all in [0,1], higher = stronger force = worse for incumbents):
  F1 = barriers_to_entry    (0 = easy entry, 1 = very high barriers)
  F2 = supplier_power       (0 = fragmented, 1 = monopoly supplier)
  F3 = buyer_power          (0 = fragmented, 1 = monopsony buyer)
  F4 = substitute_threat    (0 = no substitutes, 1 = perfect substitutes)
  F5 = rivalry_intensity    (0 = cooperative oligopoly, 1 = full price war)

Industry profitability:
  profitability = w1·F1 - w2·F2 - w3·F3 - w4·F4 - w5·F5 + base_margin
  normalized to [0,1]

Porter (1980) empirical weights (from practitioner calibration studies):
  w1 (barriers protect margin)   = 0.25
  w2 (supplier power erodes margin) = 0.20
  w3 (buyer power erodes margin)    = 0.20
  w4 (substitutes cap margin)       = 0.15
  w5 (rivalry erodes margin)        = 0.20
  base_margin                        = 0.50

  So profitability = 0.50 + 0.25·F1 - 0.20·F2 - 0.20·F3 - 0.15·F4 - 0.20·F5
```

```
Force evolution per tick:
  barriers_to_entry[t+1] = barriers[t] + investment_rate·capacity_building - entry_erosion_rate
  supplier_power[t+1]    = supplier_power[t] + concentration_delta - diversification_rate
  buyer_power[t+1]       = buyer_power[t] + buyer_concentration_delta
  substitute_threat[t+1] = substitute_threat[t] + innovation_shock - incumbent_rd_effect
  rivalry_intensity[t+1] = f(market_growth, concentration_index, exit_barriers)

Rivalry intensity Herfindahl proxy:
  If market is growing (global__gdp_growth > 0.5), rivalry decreases.
  If market is shrinking, rivalry increases.
  rivalry_delta = base_rivalry_drift - growth_effect · (global__gdp_growth - 0.5)
```

6.3 Parameter Table

Parameter	Description	Empirical Range	Default
`w_barriers`	Weight: barriers → profitability	0.15 – 0.35	0.25
`w_supplier`	Weight: supplier power → profitability erosion	0.15 – 0.30	0.20
`w_buyer`	Weight: buyer power → profitability erosion	0.15 – 0.30	0.20
`w_substitute`	Weight: substitute threat → profitability cap	0.10 – 0.25	0.15
`w_rivalry`	Weight: rivalry → profitability erosion	0.15 – 0.30	0.20
`base_margin`	Base profitability absent any force effects	0.40 – 0.60	0.50
`entry_erosion_rate`	Rate at which barriers naturally decay per tick	0.01 – 0.05	0.02
`rivalry_growth_sensitivity`	How much GDP growth dampens rivalry	0.10 – 0.40	0.25

6.4 Normalization Mapping

Real-world concept	Env key	Normalization
Barriers to entry	`porter__barriers_to_entry`	0 = open market, 1 = fortress industry
Supplier bargaining power	`porter__supplier_power`	0 = fragmented suppliers, 1 = monopoly
Buyer bargaining power	`porter__buyer_power`	0 = fragmented buyers, 1 = monopsony
Threat of substitutes	`porter__substitute_threat`	0 = no substitutes, 1 = perfect substitutes
Competitive rivalry intensity	`porter__rivalry_intensity`	0 = cooperative, 1 = price war
Industry profitability	`porter__profitability`	[0, 1] = weighted combination
Investment signal	`porter__capacity_investment`	[0, 1] = actor investment in capacity this tick

6.5 Python Class

```python
# core/theories/porter_five_forces.py
"""
Porter's Five Forces (1980)

Five force variables [0,1]:
  barriers_to_entry, supplier_power, buyer_power,
  substitute_threat, rivalry_intensity

Industry profitability:
  P = base_margin + w1·barriers - w2·supplier - w3·buyer - w4·substitute - w5·rivalry

Force evolution:
  - barriers_to_entry: raised by capacity investment, eroded by entry_erosion_rate
  - rivalry_intensity:  modulated by GDP growth (growth → less rivalry)
  - Others: driven by actor actions read from env

Reference: Porter (1980) Competitive Strategy. Free Press.
"""
from __future__ import annotations

import logging
from typing import TYPE_CHECKING

from pydantic import BaseModel, Field

from core.theories import register_theory
from core.theories.base import TheoryBase, TheoryStateVariables

if TYPE_CHECKING:
    from core.agents.base import BDIAgent

logger = logging.getLogger(__name__)


@register_theory("porter_five_forces")
class PorterFiveForces(TheoryBase):
    """
    Industry competitive structure dynamics.

    Domains: market, corporate_strategy, industry_analysis
    Priority: 0 (runs before supply/demand and contagion theories)
    """

    THEORY_ID = "porter_five_forces"
    DOMAINS = ["market", "corporate_strategy", "industry_analysis"]

    class Parameters(BaseModel):
        # Profitability weights
        w_barriers: float = Field(default=0.25, ge=0.0, le=1.0)
        w_supplier: float = Field(default=0.20, ge=0.0, le=1.0)
        w_buyer: float = Field(default=0.20, ge=0.0, le=1.0)
        w_substitute: float = Field(default=0.15, ge=0.0, le=1.0)
        w_rivalry: float = Field(default=0.20, ge=0.0, le=1.0)
        base_margin: float = Field(default=0.50, ge=0.0, le=1.0)
        # Force evolution rates
        entry_erosion_rate: float = Field(
            default=0.02, ge=0.0, le=0.2,
            description="Natural decay of barriers per tick (competitive pressure)"
        )
        rivalry_growth_sensitivity: float = Field(
            default=0.25, ge=0.0, le=1.0,
            description="How much market growth dampens rivalry intensity"
        )

    @property
    def state_variables(self) -> TheoryStateVariables:
```

```python
        return TheoryStateVariables(
            reads=[
                "porter__barriers_to_entry",
                "porter__supplier_power",
                "porter__buyer_power",
                "porter__substitute_threat",
                "porter__rivalry_intensity",
                "porter__capacity_investment",  # set by actor actions
                "keynesian__gdp_normalized",    # GDP growth for rivalry dampening
                "global__trade_volume",
            ],
            writes=[
                "porter__barriers_to_entry",
                "porter__supplier_power",
                "porter__buyer_power",
                "porter__substitute_threat",
                "porter__rivalry_intensity",
                "porter__profitability",
            ],
            initializes=[
                "porter__barriers_to_entry",
                "porter__supplier_power",
                "porter__buyer_power",
                "porter__substitute_threat",
                "porter__rivalry_intensity",
                "porter__profitability",
                "porter__capacity_investment",
            ],
        )

    def setup(self, env: dict[str, float]) -> dict[str, float]:
        inits = super().setup(env)
        # Default: moderately competitive industry at start
        defaults = {
            "porter__barriers_to_entry": 0.50,
            "porter__supplier_power":    0.40,
            "porter__buyer_power":       0.40,
            "porter__substitute_threat": 0.30,
            "porter__rivalry_intensity": 0.50,
            "porter__capacity_investment": 0.0,
        }
        for k, v in defaults.items():
            if k not in env:
                inits[k] = v
        return inits

    def update(
        self,
        env: dict[str, float],
        agents: list["BDIAgent"],
        tick: int,
    ) -> dict[str, float]:
        p = self.params

        # Read current force values
        barriers    = env.get("porter__barriers_to_entry", 0.50)
        supplier    = env.get("porter__supplier_power",    0.40)
        buyer       = env.get("porter__buyer_power",       0.40)
        substitute  = env.get("porter__substitute_threat", 0.30)
        rivalry     = env.get("porter__rivalry_intensity", 0.50)

        # Actor-driven force changes
        capacity_investment = env.get("porter__capacity_investment", 0.0)

        # barriers_to_entry: investment raises them, time erodes them
        new_barriers = max(0.0, min(1.0,
            barriers + capacity_investment * 0.10 - p.entry_erosion_rate
        ))

        # rivalry_intensity: modulated by GDP growth
```

```
        # GDP above baseline (0.5) → growing market → less rivalry
        gdp = env.get("keynesian__gdp_normalized", 0.5)
        gdp_growth_signal = gdp - 0.5  # positive = above baseline
        rivalry_delta = -p.rivalry_growth_sensitivity * gdp_growth_signal
        # Trade disruption increases rivalry (companies compete for smaller pie)
        trade = env.get("global__trade_volume", 0.5)
        if trade < 0.4:  # significant trade contraction
            rivalry_delta += (0.4 - trade) * 0.20
        new_rivalry = max(0.0, min(1.0, rivalry + rivalry_delta * 0.1))

        # supplier_power, buyer_power, substitute_threat:
        # evolve slowly without specific actor actions; small mean-reversion
        mean_reversion_rate = 0.01
        new_supplier   = supplier   + mean_reversion_rate * (0.40 - supplier)
        new_buyer      = buyer      + mean_reversion_rate * (0.40 - buyer)
        new_substitute = substitute + mean_reversion_rate * (0.30 - substitute)

        # Clamp all forces
        new_supplier   = max(0.0, min(1.0, new_supplier))
        new_buyer      = max(0.0, min(1.0, new_buyer))
        new_substitute = max(0.0, min(1.0, new_substitute))

        # Industry profitability:
        # P = base_margin + w1·barriers - w2·supplier - w3·buyer - w4·sub - w5·rivalry
        profitability = (
            p.base_margin
            + p.w_barriers   * new_barriers
            - p.w_supplier   * new_supplier
            - p.w_buyer      * new_buyer
            - p.w_substitute * new_substitute
            - p.w_rivalry    * new_rivalry
        )
        profitability = max(0.0, min(1.0, profitability))

        return {
            "porter__barriers_to_entry": new_barriers,
            "porter__supplier_power":    new_supplier,
            "porter__buyer_power":       new_buyer,
            "porter__substitute_threat": new_substitute,
            "porter__rivalry_intensity": new_rivalry,
            "porter__profitability":     profitability,
        }
```

6.6 Environment Variable Reference

Key	Type	Range	Description
`porter__barriers_to_entry`	write	[0, 1]	0 = open market, 1 = very high entry barriers
`porter__supplier_power`	write	[0, 1]	0 = fragmented suppliers, 1 = monopoly
`porter__buyer_power`	write	[0, 1]	0 = fragmented buyers, 1 = monopsony
`porter__substitute_threat`	write	[0, 1]	0 = no substitutes, 1 = perfect substitutes available
`porter__rivalry_intensity`	write	[0, 1]	0 = cooperative oligopoly, 1 = ruinous price war
`porter__profitability`	write	[0, 1]	Weighted profitability index; 0.5 = neutral margin

`porter__capacity_investment`	read	[0, 1]	Set by actor actions each tick; 0 = no investment

6.7 Key Limitations

- **Static weights.** Porter's original weights are derived from practitioner judgment, not econometric estimation. The defaults (w1=0.25, w2–w5=0.15–0.20) are reasonable starting points but should be calibrated to the specific industry in high-stakes scenarios.
- **No network effects.** Modern digital markets exhibit increasing returns and winner-take-all dynamics not captured by the five forces (Eisenmann et al. 2006). The substitute_threat variable partially captures this but not endogenously.
- **Mean reversion is artificial.** The 1% per-tick mean reversion for supplier, buyer, and substitute forces is a stability hack. Real structural change is driven by M&A, regulation, and technology — these should come from actor actions or external shocks, not the base theory.
- **No financial sub-model.** Profitability does not feed back into actor capacity. Connecting `porter__profitability` → actor budget → `porter__capacity_investment` requires an agent rule or a separate financial theory module.

---

7. Theory Composition

7.1 Domain-Theory Map

```python
# From forge/theory_mapper.py
DOMAIN_THEORY_MAP: dict[str, list[str]] = {
    "geopolitics": [
        "richardson_arms_race",   # priority 0: sets military readiness
        "wittman_zartman",        # priority 1: reads readiness, outputs ripeness
        "fearon_bargaining",      # priority 1: reads readiness, outputs conflict P
    ],
    "market": [
        "porter_five_forces",     # priority 0: industry structure
        "supply_demand_shock",    # priority 1: (future module)
        "market_contagion",       # priority 2: (future module)
    ],
    "macro": [
        "keynesian_multiplier",   # priority 0: fiscal propagation
        "regulatory_shock",       # priority 1: (future module)
        "trade_flow_gravity",     # priority 1: (future module)
    ],
    "org": [
        "principal_agent",        # (future module)
        "diffusion_of_innovation",# (future module)
        "institutional_inertia",  # (future module)
    ],
}
```

7.2 Conflict Table

```python
# Theories that cannot compose — TheoryMapper picks the higher-scoring one
THEORY_CONFLICTS: list[tuple[str, str]] = [
    ("wittman_zartman", "fearon_bargaining"),
    # Both output a conflict/negotiation probability for the same actors.
    # Running both double-counts war onset/termination dynamics.
    # Rule: use fearon_bargaining for onset-focused scenarios,
```

```
    #       wittman_zartman for termination/mediation scenarios.
]
```

7.3 Priority Table

```
THEORY_PRIORITY: dict[str, int] = {
    "richardson_arms_race": 0,   # must run first: sets military readiness
    "keynesian_multiplier": 0,   # must run first: sets GDP and fiscal state
    "porter_five_forces":   0,   # must run first: sets market structure
    "fearon_bargaining":    1,   # reads military readiness (priority 0 output)
    "wittman_zartman":      1,   # reads military balance + war costs
}
```

Ordering rationale: Priority 0 theories update the physical/structural state (arms, GDP, market structure). Priority 1 theories compute probabilities and equilibria that depend on that updated state. This ensures Fearon and Zartman read the correct post-Richardson military balance.

7.4 Compatible Theory Combinations

Combination	Compatible	Notes
`richardson + fearon_bargaining`	Yes	Classic: arms race → conflict probability
`richardson + wittman_zartman`	Yes	Arms race → ripeness for settlement
`richardson + keynesian_multiplier`	Yes	Arms spending creates fiscal shock
`fearon_bargaining + wittman_zartman`	**No**	THEORY_CONFLICTS: both model conflict probability
`porter_five_forces + keynesian_multiplier`	Yes	GDP growth dampens rivalry via cross-read
`porter_five_forces + richardson`	Possible	Use if scenario involves defense industry; no shared keys
All five	**No**	fearon/wittman conflict prevents running all simultaneously

7.5 Cross-Theory Environment Key Dependencies

```
┌──────────────────────────────────────────────────────────────────┐
│  Cross-theory env key flow                                       │
│                                                                  │
│  richardson_arms_race                                            │
│    WRITES → actor_a__military_readiness                          │
│             actor_b__military_readiness                          │
│             richardson__escalation_index                         │
│                   │                                              │
│                   ▼                                              │
│  fearon_bargaining                                               │
│    READS  ← actor_a__military_readiness (→ win_prob_a)           │
│    WRITES → fearon__win_prob_a                                   │
│             fearon__conflict_probability                         │
```

```
│                  │                                                │
│                  ▼                                                │
│  wittman_zartman                                                  │
│    READS  ← fearon__win_prob_a                                    │
│             fearon__war_cost_a / war_cost_b                       │
│    WRITES → zartman__eu_war_a / eu_war_b                          │
│             zartman__mhs / ripe_moment                            │
│             zartman__negotiation_probability                      │
│                                                                   │
│  keynesian_multiplier                                             │
│    READS  ← global__trade_volume                                  │
│    WRITES → keynesian__gdp_normalized                             │
│             keynesian__unemployment                               │
│                   │                                               │
│                   ▼                                               │
│  porter_five_forces                                               │
│    READS  ← keynesian__gdp_normalized (rivalry dampening)         │
│             global__trade_volume (rivalry increase on disruption) │
│    WRITES → porter__profitability                                 │
│             porter__rivalry_intensity                             │
└───────────────────────────────────────────────────────────────────┘
```

---

8. Full Architecture Diagram

```
╔═══════════════════════════════════════════════════════════════════╗
║  Crucible SimRunner — Theory Module Architecture                  ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║  SHARED ENVIRONMENT  env: dict[str, float]   (all values in [0, 1])  ║
║  ═══════════════════════════════════════════════════════════    ║
║                                                                   ║
║  actor_a__military_readiness ──────────────────────────┐         ║
║  actor_b__military_readiness ──────────────────────────┼──┐      ║
║  richardson__escalation_index                           │  │      ║
║  richardson__stable                                     │  │      ║
║                                                         │  │      ║
║  fearon__win_prob_a ────────────────────────────────────┘  │      ║
║  fearon__win_prob_b_estimate                               │      ║
║  fearon__conflict_probability ──────────────────────────────┘      ║
║  fearon__settlement_range_width                                     ║
║  fearon__war_cost_a / war_cost_b                                    ║
║                                                                   ║
║  zartman__eu_war_a / eu_war_b                                      ║
║  zartman__mhs / ripe_moment                                         ║
║  zartman__negotiation_probability                                   ║
║  zartman__stalemate_duration                                        ║
║  zartman__mediator_present  ←─── external shock / agent action     ║
║                                                                   ║
║  keynesian__gdp_normalized ──────────────────────────┐         ║
║  keynesian__fiscal_shock_pending ←── sanctions shock      │         ║
```

```
‖  keynesian__multiplier                                  |            ‖
‖  keynesian__unemployment                                |            ‖
‖                                                         |            ‖
‖  porter__barriers_to_entry                              |            ‖
‖  porter__supplier_power                                 |            ‖
‖  porter__buyer_power                                    |            ‖
‖  porter__substitute_threat                              |            ‖
‖  porter__rivalry_intensity ←──────────────────────────┘            ‖
‖  porter__profitability                                                ‖
‖                                                                      ‖
‖  global__trade_volume  ←─── both keynesian + porter read this        ‖
‖  global__urgency_factor ←── zartman reads this                       ‖
‖                                                                      ‖
‖═══════════════════════════════════════════════════════════════════════‖
‖                                                                      ‖
‖  TICK LOOP (SimRunner)                                               ‖
‖  ═══════════════════════════════════════════════════════════       ‖
‖                                                                      ‖
‖  ┌──────────────────────────────────────────────────────────┐     ‖
‖  │ PRIORITY 0 (runs first, in parallel-safe delta accumulation)   │     ‖
‖  │                                                          │     ‖
‖  │  ┌──────────────────┐  ┌──────────────────┐            │     ‖
‖  │  │ RichardsonArmsRace  │  │ KeynesianMultiplier   │            │     ‖
‖  │  │                  │  │                  │            │     ‖
‖  │  │  dx/dt = k·y-a·x+g  │  │  M = 1/(1-MPC(1-t)+m)│            │     ‖
‖  │  │  dy/dt = l·x-b·y+h  │  │  GDP += M·shock·dt   │            │     ‖
‖  │  │                  │  │  Okun → unemployment  │            │     ‖
‖  │  └──────────────────┘  └──────────────────┘            │     ‖
‖  │                                                          │     ‖
‖  │  ┌──────────────────┐                                    │     ‖
‖  │  │ PorterFiveForces    │                                    │     ‖
‖  │  │                  │                                    │     ‖
‖  │  │  P = f(barriers,    │                                    │     ‖
‖  │  │    -supplier,       │                                    │     ‖
‖  │  │    -buyer,          │                                    │     ‖
‖  │  │    -substitute,     │                                    │     ‖
‖  │  │    -rivalry)        │                                    │     ‖
‖  │  └──────────────────┘                                    │     ‖
‖  └──────────────────────────────────────────────────────────┘     ‖
‖          │ delta_0 applied to env                                    ‖
‖          ▼                                                           ‖
‖  ┌──────────────────────────────────────────────────────────┐     ‖
‖  │ PRIORITY 1 (reads priority-0 outputs)                        │     ‖
‖  │                                                          │     ‖
‖  │  ┌──────────────────┐  ┌──────────────────┐            │     ‖
‖  │  │ FearonBargaining    │  │ WittmanZartman       │            │     ‖
‖  │  │  [onset scenario]   │  │  [termination scen.] │            │     ‖
‖  │  │                  │  │                  │            │     ‖
‖  │  │  win_prob ← mil     │  │  EU_war = p·V - c_A  │            │     ‖
‖  │  │  |p-p̂| > c_A+c_B   │  │  MHS if EU < floor   │            │     ‖
‖  │  │  → conflict_prob    │  │  ripe → negotiate_P  │            │     ‖
‖  │  └──────────────────┘  └──────────────────┘            │     ‖
‖  │        (mutually exclusive — THEORY_CONFLICTS enforces)        │     ‖
‖  └──────────────────────────────────────────────────────────┘     ‖
```

```
│          │ delta_1 applied to env                                    │
│          ▼                                                          │
│     record_metrics(env, tick)  →  MetricRecord                      │
│     snapshot if tick % snap_interval == 0  →  SimSnapshot           │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

---

9. File Layout

```
core/
└── theories/
    ├── __init__.py                  ← registry: register_theory, get_theory, list_theories
    ├── base.py                      ← TheoryBase ABC, TheoryStateVariables
    ├── richardson_arms_race.py      ← Theory 1
    ├── fearon_bargaining.py         ← Theory 2
    ├── wittman_zartman.py           ← Theory 3
    ├── keynesian_multiplier.py      ← Theory 4
    └── porter_five_forces.py        ← Theory 5

forge/
└── theory_mapper.py                 ← DOMAIN_THEORY_MAP, THEORY_CONFLICTS,
                                       THEORY_PRIORITY, TheoryMapper
```

All five theory files must be imported somewhere before `SimRunner.setup()` runs, so the `@register_theory` decorators fire and populate `_THEORY_REGISTRY`. Recommended: import all five in `core/theories/__init__.py` at the bottom of the file, after the registry functions are defined.

```python
# core/theories/__init__.py  (bottom of file, after registry definitions)
# Auto-register all built-in theories
from core.theories import (  # noqa: E402, F401
    richardson_arms_race,
    fearon_bargaining,
    wittman_zartman,
    keynesian_multiplier,
    porter_five_forces,
)
```

---

10. Quick Reference: Equations at a Glance

Theory	Core Equation	Stability / Trigger
Richardson	`dx/dt = k·y - a·x + g`	`a·b > k·l` for stability
Fearon	Settlement iff `|p - p̂_B| ≤ c_A + c_B`	War if info gap or power shift exceeds cost cushion
Zartman	`EU_war(A) = p·V - c_A`	Ripe iff MHS AND duration ≥ min AND mediator/urgency

Keynesian	`M = 1 / (1 - MPC·(1-t) + m)`	`M > 1` always; valid if denominator > 0
Porter	`P = base + w1·F1 - w2·F2 - w3·F3 - w4·F4 - w5·F5`	Profitability ∈ [0,1] by construction

Document version: 1.0 — Week 1 architecture phase. Theory stubs to be implemented Week 2.